CONCURRENCY

1 Mutual exclusion, deadlocks and starvation

As indicated earlier, a contemporary computer system will not limit itself to the simultaneous execution of a single process, but will manage a whole series of processes and threads. In the context of a single processor, we speak of *multiprogramming*, whereas in the case of multiple processors, we use the term *multiprocessing*. In both cases, these processes need to share many common *resources*, such as I/O devices, memory, processor time, etc. It is often also important that multiple processes cannot have access to a resource at the same time, in order to guarantee the correct running of the programs. Take a look at this simple example:

```
procedure echo;
var outp, inp: character;
begin
   input (inp, keyboard);
   outp := inp;
   output (outp,display);
end
```

We assume several processes will use this simple procedure and we place it in memory shared by all processes, so we only need to store a single copy of the procedure in memory. This also means that the variables inp and outp are also shared by all processes. Now suppose a first process P0 calls the *echo* procedure and is interrupted just after reading in the variable inp. When a second process P1 also calls the procedure echo, the variable inp will be overwritten. If still later process P0 is resumed, the wrong character will appear on the screen. Note that the same problem can occur if P0 is not interrupted, but there is a second processor on which process P1 is active.

In this example we see that the *interleaving* of both processes leads to an incorrect course, namely, the first process P0 is interrupted at an unfortunate moment. To guarantee a correct operation, a following process may only use the procedure *echo*, if the previous call is completely finished. This procedure is therefore called a *critical section* of the process. The fact that several processes are present simultaneously in the critical section is usually referred to as *mutual exclusion*. In addition to the potential problems that can arise from multiple processes entering a critical section simultaneously, other problems can arise.

For example, when two processes P0 and P1 both need resources R0 and R1 for a certain operation. Then it can happen that P0 can dispose of R0, while P1 already has resource R1 at its disposal. Both processes are blocked and wait for each other, but neither of them will ever continue unless the other releases his resource. In this case we speak of a *deadlock*. Finally it can also happen that three (or more) processes share a resource and that two of them can take turns in getting the resource. In this case the third process never executes and we speak of *starvation*. It is the task of the operating system to ensure that all processes execute correctly regardless of the speed at which other processes are executing.

There are various techniques for realising mutual exclusion. The problem can be tackled from a software point of view by stating that the processes themselves must be responsible for realising mutual exclusion. Hardware-based solutions can also be developed. Finally a solution can be offered via the operating system or the programming language used by the user. We will start by looking at some software-based solutions, as these have historically been the first to appear.

2 Software-based solutions

The first techniques for realizing mutual exclusion between two processes were developed by Dekker in the 1960s. In this section we discuss Dekker's algorithm by arriving at a solution step by step. We then look at a simpler solution devised by Peterson in the early 1980s.

2.1 Dekker's algorithm

When we think about how to realize mutual exclusion, the solution seems very obvious. For each critical section, we keep a global variable that indicates whether a process is currently present in the critical section. If this variable indicates that a process is present, all other processes must wait. If the critical section is free, then the process can enter the section after modifying the global variable. Finally, when leaving the section, the process will again modify the variable to indicate that the section is free again.

However, this gives rise to problems: to enter the section the process must first test if the section is free and if so it will adjust the variable and enter. However, if the process is interrupted exactly after it sees that the section is free, then another process could also start the same procedure and find that the section is free. Since the first process has not yet modified the variable, both processes may end up in the critical section. In short, because a process can be interrupted between the test and the write operation, mutual exclusion is not guaranteed.

There is also a second disadvantage: if the section is occupied, the waiting processes must regularly check whether the section has been freed up in the meantime. This repeated checking takes processor time and is called *busy waiting.*

First attempt: A first idea is to use a global variable turn which indicates which of the two processes may enter the critical section next. So as long as turn indicates that it is the other process' turn, we must wait busy until the section becomes free. When a process leaves the section the turn will go to the other process (so the variable **turn** is modified):

PROCESS 0	**PROCESS 1**
..	..
while turn 6= 0 **do** {nothing};	**while** turn 6= 1 **do** {nothing};
<critical section>	<critical section>
turn := 1;	turn := 0;
..	..

Although this solution is correct, it has some serious drawbacks. First, both processes must take turns entering the section, which causes the faster process, which needs to be in the section a lot, to be delayed. In addition, the variable turn must also be initialized, which favors one of the two processes.

Second and third attempt: Instead of using a single global variable, we can also try to keep a variable for each of the two processes (flag[0] and flag[1]). To enter the critical zone, the process first checks if the other process also has this intention by looking at its variable. If this is not the case, we set the flag of this process to true and enter the critical section. On leaving the section, the process sets its flag back to false. The global variables are initialized to false:

PROCESS 0

```
..
while flag[1] do {nothing};
flag[0]:=true;
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
while flag[0] do {nothing};
flag[1]:=true;
<critical section>
flag[1]:=false;
..
```

This solution has serious shortcomings: when a process is interrupted just after the while loop, the other process can also complete the while loop as the flag of the first process still indicates false. Thus, there is no mutual exclusion. Also swapping the while loop and the assignment does not help:

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do {nothing};
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do {nothing};
<critical section>
flag[1]:=false;
..
```

So in this case we first indicate that we want to enter the critical section, before checking if the other one has this intention too. In this case, a deadlock can occur when a process is interrupted right after it sets its flag to true, allowing the other process to do the same and both processes are stuck in the while loop.

Fourth attempt: The solution seems to present itself now: when we notice that the other process has also set its flag, we set our flag back to false and wait a while before setting it back to true. This way we give the other process the chance to enter the critical section:

PROCESS 0

```
..
flag[0]:=true;
while flag[1] do
   flag[0]:=false;
   <delay a short time>
   flag[0]:=true;
end;
<critical section>
flag[0]:=false;
..
```

PROCESS 1

```
..
flag[1]:=true;
while flag[0] do
   flag[1]:=false;
   <delay a short time>
   flag[1]:=true;
end;
<critical section>
flag[1]:=false;
..
```

This solution is not ideal because there is no upper limit on the waiting time of both processes. It can happen that, by a very unfortunate coincidence, both processes want to give priority to each other: they set their flags to false shortly after each other, wait a moment to let the other one go ahead, and then set their flags to true again shortly after each other, to indicate that they both want to enter the critical section.

Final solution: The solution consists in adding a single variable turn indicating which of the two processes will be courteous if it turns out that both have set their flag to true (turn = 1 indicates that process 0 is courteous).

Let us check the correctness of this code in a rather formal way. As precondition to the critical section of process P0 we can see that

$$flag[0] \wedge (\neg flag[1] \vee (turn=0)),$$

while the precondition for P1 is that

$$flag[1] \wedge (\neg flag[0] \vee (turn=1)).$$

These two conditions cannot occur simultaneously, thus guaranteeing mutual exclusion.

```
PROCESS 0
..
flag[0]:=true;
while flag[1] do
 if turn = 1 then
    flag[0]:=false;
    while turn = 1 do {nothing};
    flag[0]:=true;
 end;
end;
<critical section>
turn:=1;
flag[0]:=false;
..
```

```
PROCESS 1
..
flag[1]:=true;
while flag[0] do
 if turn = 0 then
    flag[1]:=false;
    while turn = 0 do {nothing};
    flag[1]:=true;
 end;
end;
<critical section>
turn:=0;
flag[1]:=false;
..
```

Deadlocks cannot occur either, namely, suppose process P0 gets caught in the inner while loop, then P1's flag must be true and turn = 1. In short, P1 is or has the intention to enter the critical section. If P1 is already in this zone, then it will set turn to zero when it exits, eventually freeing P0 from the loop. If P1 has yet to enter the critical zone, it can do so without any problem since turn equals 1 and the flag of P0 equals false. Again, P0 will be able to leave the loop after P1 leaves the critical section and sets turn to zero. Note that when the two processes both wish to enter the critical section, the one that had the least recent access to the section will take precedence.

2.2 Peterson's algorithm

In 1981 Peterson proposed a more elegant solution to the problem of the two processes. The idea is that a process first announces that it wants to enter the critical zone by setting its flag to true and then sets the turn equal to the other process. Then it tests if the other process wants to enter the critical zone. If not the process can enter the section by itself, otherwise it waits until the turn variable is equal to its number. Thus, if the two processes both wish to enter the critical zone, the process that last changed the turn variable will give priority to the other process. Thus, the process that reaches the while loop first wins, which also guarantees that the other process will gain access before the same process can enter the section again. Of course, if only one process wants to use the critical section, it can do so many times without the other process taking its turn.

```
PROCESS 0
..
flag[0]:=true;
turn = 1;
while flag[1] and turn = 1 do {nothing };
<critical section>
flag[0]:=false;
..
```

Peterson's algorithm can also be generalised more easily to n processes compared to Dekker's algorithm. This generalisation consists of the fact that each process has to pass through n - 1 phases before it can enter the critical section. Thus, for n = 2, we have only one phase that corresponds exactly to the three lines of code. The code for n processes uses an array flag, where flag[i] indicates which phase process i is in (it is zero if the process does not wish to enter the critical section). There is also an array of n - 1 elements called turn, where turn[j] indicates which process was last to execute the second line out of phase. A process will turn to the next phase if no other process is in this phase or if another process entered the phase later (note that there are no problems when the process is interrupted in the middle of the wait test):

```
PROCESS i
..
for j = 1 to n-1 do
   flag[i]:= j;
   turn[j] := i;
   wait until (∀ k 6= i, flag[k] < j) or (turn[j] 6= i);
end;
<critical section>
flag[i]:=0;
..
```

In case of n > 2 processes a process can be passed an unlimited number of times by other processes. Although this algorithm is simple, it still uses busy waiting, i.e., the processes must repeatedly check if they can proceed one stage further. This requires a lot of processor effort that is lost. This is why hardware solutions have been developed.

3 Hardware solutions

A first simple hardware solution consists of preventing interrupts when a process is in a critical section. This is a working solution, but when the time needed to finish the critical section is large, or when the process has to wait for an event during its critical section, this leads to a very uneven and inefficient use of the processor. In addition, this only solves the problem in the multiprogramming scenario and not in the case of multiple processors.

A better option is to use special machine instructions. Two important examples are the *test and set* instruction and the *exchange* instruction. The first instruction allows a test followed by a write operation to be performed as a single instruction and thus as an atomic operation that cannot be interrupted. Thinking back to our first reflections on concurrency, we wished to use a single global variable to protect a critical section. The problem with this was that after a process checked whether the section was occupied, it could be interrupted, allowing other processes to undergo the same test before the former process had claimed the section. With the test and set instruction we can now use this simple approach for any number of processes. Initially we set the variable bolt to false, before a process enters the critical section it first tests if bolt is false, and if so, it sets it to true. Upon exiting the critical zone, bolt is set to false again. In the code below, the test set instruction will return the value of the bolt variable at the start of the instruction:

```
PROCESS i
..
while (testset(bolt)) do {nothing};
<critical section>
bolt:=false;
..
```

A similar solution can be obtained by the exchange instruction. This instruction allows the contents of a register and a memory location to be exchanged. The idea for mutual exclusion is to give each process a local variable key, which is initialised to zero. In addition, there is a single global variable **bolt**, which we initialize to 1. When a process now wishes to enter the critical section, it first performs an exchange on the key and bolt variable until the key is equal to 1. When a process performs this operation while another process is in the critical section, it will only exchange two zeros. When leaving the section, another exchange operation is performed.

Both solutions correspond to a so-called *spinlock*, are very simple, applicable to many processes and critical zones and function both in a multiprogramming and multiprocessor context (although an implementation of the test set instruction in a multiprocessor environment does not seem obvious). Nevertheless, it also has some serious drawbacks. First of all, busy waiting is used again. Next, there is no guarantee that no starvation can occur, a process can be excluded for an arbitrary time because other processes keep claiming the critical zone. Finally, deadlock can even occur when the processor uses strict priorities in scheduling the processes (or threads).

4 Semaphores

Dijkstra made a big progress in the field of concurrency with the development of semaphores. A semaphore can be considered as a signal that allows processes to coordinate. A process can send such a signal by means of the *signal* operation, furthermore it can receive a signal by means of the *wait* operation. When the signal is not yet sent, the process will be blocked until another process sends the corresponding signal. A semaphore can be seen as a *record* consisting of a counter and a list of processes. Three operations can be performed on the counter:

- The counter can be initialized to a non-negative value.
- The counter of the semaphore s can be decreased by one via the *wait(s)* operation. If the counter gets a negative value, the process which performed the operation will be blocked and added to the list of blocked processes.
- The counter can be increased by one via the *signal(s)* operation. When the obtained value of the counter is not positive, a process is also removed from the list, or even unblocked.

Note that the counter keeps track of how many more processes can receive a signal without being blocked. If the counter is negative, its absolute value indicates the number of blocked processes. Figure 53 shows all this again in pseudo code.

Besides these semaphores, sometimes called *counting semaphores*, there also exist binary semaphores that differ from the previous ones because the counter can only take the values 0 and 1. A *waitB* signal checks whether the counter is at 1, if so we lower it to zero and continue, otherwise the process is blocked. A *signalB* call will, if the **s.queue** is empty, set the counter to 1 and otherwise unblock a process from the list. Although binary semaphores seem less general, one can prove that they are as powerful as general semaphores (we may demonstrate this during the exercises). The idea is to create a new counter as a global variable and to protect the access to it with binary semaphores. The importance of binary semaphores is that they are easier to implement/support in certain hardware architectures.

```
type semaphore = record
          count: integer;
          queue: list of process
       end;

var s: semaphore;

wait(s);
 s.count := s.count - 1;
 if s.count < 0
   then begin
     place this process in s.queue;
     block this process
   end;

signal(s);
 s.count := s.count + 1;
 if s.count ≤ 0
   then begin
     remove a process P from s.queue;
     place process P on the ready list
   end;
```

Fig. 53: Definition of a semaphore

4.1 Mutual exclusion with semaphores

Mutual exclusion is easily achieved by using semaphores. For each critical zone we define a semaphore which we initialize to 1. Whenever a process wants to enter a critical zone, it first performs the *wait(s)* operation, where s is the semaphore belonging to the critical section. When leaving the section the *signal(s)* operation is executed. Now when several processes want to enter the critical section, only one process will succeed as the others are all blocked by the *wait(s)* operation. A necessary requirement here is that the *wait* and *signal* operations cannot be interrupted or that at least no two processes can execute a wait or signal operation simultaneously, otherwise the mutual exclusion property can be lost.

4.2 The producer/consumer problem with semaphores

A common problem with concurrent processes is the so-called *producer/consumer* problem. In this problem there are one or more processes that generate data and write them to a buffer, while another process wants to have these data at its disposal and reads them from the buffer. For simplicity, we assume that the buffer is infinitely large. The required adaptation for a finite buffer is quite simple. The producer and consumer functions could look like this:

```
producer:
repeat
   produce item v;
   b[in] := v;
   in := in +1;
forever;
```

```
consumer:
repeat
   while in ≤ out do {nothing};
   w := b[out];
   out := out +1;
   consume item w;
forever;
```

We can see that on the one hand we must protect the global variables in and out, and also prevent the buffer from being called by several processes at the same time. Finally, the buffer may not be read when it is empty. For simplicity we set n equal to in - out. The use of two semaphores is obvious: a first s that must guarantee the mutual exclusion of the buffer and the counter n, and a second **ne** that indicates when the buffer contains something:

```
producer:
repeat
   produce;
   waitB(s);
   take;
   append;
   n := n +1;
   signalB(s);
   if n = 1 then signalB(ne);
   signalB(s);
forever;
```

```
consumer:
waitB(ne);
repeat
   waitB(s);
   take;
   n := n - 1;
   signalB(s);
   if n = 0 then waitB(ne);
   consume;
forever;
```

This solution seems to be correct, but there is a problem. The consumer has to perform a *waitB* operation on the semaphore **ne** if he decreased the counter n to zero. However, he cannot do this before he has executed the *signalB(s)* operation, otherwise there will be a deadlock. This may interrupt the consumer before it can consult the value of n. If a producer adds an element in between, the consumer can subsequently fetch two elements from the buffer, while only one is present. This can be solved by storing the value of n in a local variable m so that the producer procedure cannot change it.

It is also important to note that the presence of the **ne** semaphore makes this solution unusable if multiple consumers are active. For example, if 10 items are produced and two consumers each want to read five items, it is possible that only one of the consumers succeeds (because a *signal(ne)* operation may only be performed once).

The previous solution becomes even simpler with general semaphores. Now we can define next to the s semaphore also n as semaphore. This gives rise to the following code:

```
producer:
repeat
   produce;
   wait(s);
   append;
   signal(s);
   signal(n);
forever;
```

```
consumer:
waitB(ne);
repeat
   wait(n);
   wait(s);
   take;
   signal(s);
   consume;
forever;
```

The order of the two wait operations is of crucial importance. If their order was reversed, a deadlock could occur because a consumer in the critical zone might be blocked. Can we also use this code if multiple consumers are active?

4.3 Implementing semaphores

The success of semaphores depends on preventing multiple processes from simultaneously executing a wait or signal operation. With the test and set instruction we can easily achieve this by adding a variable flag to the record in Figure 53 which is initialized to false. Both the wait and signal operations will then start with a repeat that contains no instructions until test set(s.flag) is false. At the end of both operations the variable s.flag should be set back to false (as well as when a process is blocked).

We are dealing with a form of busy waiting, but the *wait* and *signal* instruction are often of very short duration. In addition, busy waiting is sometimes faster than signaling (because the process remains in the ready queue). This is especially the case when there are multiple processors, because then the waiting process will only stay busy for a few cycles, because another process can release the semaphore from another processor.

In order to realize a fast implementation of a semaphore, often a single pointer will be added to the *process control block* (see Chapter 1) of each process. The list of blocked processes is then supported by having the semaphore itself point to the first waiting process. This process will point via the pointer in the control block to the second waiting process, and so on. A single pointer per process is sufficient as a process never has to wait simultaneously on multiple semaphores. Furthermore the semaphore will usually have a pointer pointing to the last waiting process so new processes blocked by the semaphore can be quickly added to the list. In well written implementations often only a dozen instructions will be needed in a *wait* or *signal* operation. This also shows why the use of busy waiting is acceptable in this context.

During the execution of the wait of signal operation, one will also temporarily disable the interrupts so that they are always completed. Note that the temporary disabling of interrupts is sufficient on a system equipped with a single processor. In case of multiple processors, the combination with a hardware instruction such as the test and set instruction is necessary.

Temporarily disabling interrupts can have consequences for the operating system's clock. A clock is often implemented by generating an interrupt at regular intervals which increases the clock by one. Suppose, however, that the interrupts are disabled and that in the meantime multiple interrupts are received which all have the same goal: to increase the clock by one. Because these interrupts are all of the same type, when the interrupts are allowed again, the processor will simply notice that there was an interrupt of this type and will call the corresponding interrupt handler once. It is therefore not kept track of how many times a particular interrupt was generated, which can cause the clock to lose a few *ticks*.

5 The readers/writers problem

One of the most fundamental concurrency problems is the so-called *Readers/Writers* problem. This problem states that there are a number of readers that want to read data (e.g. the library catalogue), but the data can also be modified by writers. Readers can easily access the data at the same time, however, when a writer wants to modify the data, he must be the only one to access the data, so no reader will see nonsensical results. We will look at two solutions which use semaphores. In the first, simplest, solution we give priority to the readers: as long as data is read, the writers must wait.

How can we implement this? We clearly need to keep track of the number of readers, since a writer needs to know if there are still readers present. This cannot be done simply by using the **s.count** of a semaphore, as more readers means more waiting time for the writers. A global variable **rcount** seems appropriate with an associated semaphore x to guarantee the mutual exclusion of this variable. Furthermore we need a semaphore **wsem** which prevents the writers from writing as long as there are readers (this semaphore is initialized to one, like the other semaphores). When should

we use this semaphore? When a reader decreases the counter **rcount** to 0, we give a *signal(wsem)* indicating the writers can access the data. However, multiple readers can come and read for a while before a writer comes. So when we only give this *signal(wsem)* we get problems. The solution is to have readers also perform a *wait(wsem)* when they increase the counter **rcount** to one. A reader will only block if there is actually a process writing:

```
procedure reader:
 wait(x);
   rcount := rcount+1;
   if rcount = 1 then wait(wsem);
 signal(x);
 READ
 wait(x);
   rcount := rcount-1;
   if rcount = 0 then signal(wsem);
 signal(x);

procedure writer:
 wait(wsem);
   WRITE;
 signal(wsem);
```

Of course this solution can cause starvation in the writer processes. As long as new readers are added and rcount remains greater than zero, no writer process can change data. Suppose we want to give priority to the writers. We still need the rcount, because a writer still needs to wait until the readers, which are already reading, are finished. In addition, we should also keep a counter wcount that indicates how many writers are active and a semaphore y to control access to this counter. The counter wcount is used to check when the readers are allowed again. It is adapted like the rcount counter. An extra semaphore rsem now indicates when the readers can start reading again. Unlike the wsem, the rsem semaphore will make sure no readers can start. Of course it should not prevent readers from reading simultaneously.

```
procedure reader:
 wait(rsem);
  wait(x);
   rcount := rcount+1;
   if rcount = 1 then wait(wsem);
  signal(x);
 signal(rsem);
READ
wait(x);
 rcount := rcount-1;
 if rcount = 0 then signal(wsem);
signal(x);


procedure writer:
 wait(y);
   wcount := wcount+1;
   if wcount = 1 then wait(rsem);
 signal(y);
 wait(wsem);
   WRITE;
 signal(wsem);
 wait(y);
   wcount := wcount-1;
   if wcount = 0 then signal(rsem);
 signal(y);
```

This code has a small shortcoming: suppose during writing many readers appear. Then they all end up in the queue of the semaphore **rsem**. When the writing action is completely finished, the readers can start reading. If a writer arrives shortly after that, it must precede the entire queue in **rsem**. In short, we have a priority queue where high priority clients that find the queue empty have to let all low priority clients pass, which is not desirable. We can avoid this by introducing another semaphore *z* and placing the operations wait(z) and signal(z) around the *wait(rsem)* and *signal(rsem)* instructions in the reader procedure.